

```

import numpy as np
#Linear Search
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# contoh
data = [10, 20, 30, 40, 50]
print(linear_search(data, 30)) # output: 2

```

2

```

# Binary Search (Iteratif)
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# contoh (data harus terurut)
data = [10, 20, 30, 40, 50]
print(binary_search(data, 40)) # output: 3

```

3

```

Binary Search (Rekursif)
def binary_search_recursive(arr, left, right, target): # kondisi berhenti (tidak ditemukan)
    if left > right:
        return -1
    mid = (left + right) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, mid + 1, right, target) # jika target lebih besar
    else:
        return binary_search_recursive(arr, left, mid - 1, target) # jika target lebih kecil

# contoh
data = [10, 20, 30, 40, 50]
print(binary_search_recursive(data, 0, len(data)-1, 20)) # output: 1 # ke kiri

```

1

```

# Jump Search
import math
def jump_search(arr, target):# jumlah elemen
    n = len(arr)           # ukuran lompatan √n
    step = int(math.sqrt(n)) # indeks awal blok
    prev = 0
    # lompat blok
    while arr[min(step, n) - 1] < target:# cek ujung blok
        prev = step                # pindah ke blok berikutnya
        step += int(math.sqrt(n))  # tambah langkah
        if prev >= n:               # jika melebihi array
            return -1

    # blok linier
    while prev < n and arr[prev] < target:# cek satu per satu dalam blok
        prev += 1                  # jika ditemukan

    if prev < n and arr[prev] == target:
        return prev
    # The previous 'return -1' here made the following 'for' loop unreachable
    # It has been removed to allow the 'linear search di blok' to execute.

    # linear search di blok
    for i in range(prev, min(step, n)):
        if arr[i] == target:
            return i
    return -1          # jika tidak ditemukan

# contoh
data = [10, 20, 30, 40, 50, 60, 70]
print(jump_search(data, 60)) # output: 5

```

5

```

# Interpolation Search
def interpolation_search(arr, target):# indeks awal
    low = 0                          # indeks akhir
    high = len(arr) - 1
    while low <= high and target >= arr[low] and target <= arr[high]:
        if low == high: # jika hanya satu elemen
            if arr[low] == target: # hitung posisi perkiraan (interpolasi)
                return low
            return -1

        # Handle division by zero if arr[high] == arr[low]
        if arr[high] == arr[low]:
            if arr[low] == target:
                return low
            else:
                return -1

        # rumus interpolasi
        pos = low + ((target - arr[low]) * (high - low) //

```

```
        (arr[high] - arr[low]))
    if arr[pos] == target: # jika ditemukan
        return pos
    elif arr[pos] < target:
        low = pos + 1 # jika target lebih besar
    else:
        high = pos - 1 # jika target lebih kecil
    return -1 # jika tidak ditemukan

# contoh
data = [10, 20, 30, 40, 50]
print(f"Index of 30 in data: {interpolation_search(data, 30)}")
print(f"Index of 50 in data: {interpolation_search(data, 50)}")
print(f"Index of 15 in data: {interpolation_search(data, 15)}") # Not found
```

```
Index of 30 in data: 2
Index of 50 in data: 4
Index of 15 in data: -1
```